

Competitive Programming

Section 70: Bounded And Minimum-Cost Flows

Competitive Programming Course

Table of contents

1	From lower bounds to imbalance	3
2	Feasible circulation reduction	4
3	Minimum-cost augmenting paths	5
4	Modeling quotas and convex costs	6
5	Worked trace: lower bounds create a routing problem	7
6	Worked trace: why a residual reverse edge matters	8
7	Network-design checklist	9
8	C++ and Python implementation notes	10
9	Proof habits	11
10	Complexity and failure modes	12
11	Exercises	13

1 From lower bounds to imbalance

Ordinary flow starts every edge at zero. A lower bound $\text{low}(e)$ forbids that. Pretend the mandatory part has already been sent and write $\text{flow}(e) = \text{low}(e) + \text{extra}(e)$. The extra flow has capacity $\text{high}(e) - \text{low}(e)$, but the mandatory parts leave vertex imbalances.

For every mandatory low on $u \rightarrow v$, subtract it from $\text{balance}[u]$ and add it to $\text{balance}[v]$. A positive balance means the vertex has already received too much and must send that amount through optional residual edges; a negative balance must receive flow.

2 Feasible circulation reduction

Add a super-source SS and super-sink TT :

- if $\text{balance}[v] > 0$, add $SS \rightarrow v$ with that capacity;
- if $\text{balance}[v] < 0$, add $v \rightarrow TT$ with $-\text{balance}[v]$ capacity.

A bounded circulation exists exactly when a maximum flow saturates every edge out of SS . Necessity follows by subtracting the lower bounds from any valid circulation. Sufficiency follows by adding them back to a saturating auxiliary flow.

For a bounded $s \rightarrow t$ flow, add a return edge $t \rightarrow s$; fixing its lower and upper bounds to F asks for exactly F units.

3 Minimum-cost augmenting paths

In a residual graph, a forward edge costs c and its reverse costs $-c$. The reverse edge is essential: it lets a later augmentation repair an earlier choice. Repeatedly augmenting on a cheapest residual $s \rightarrow t$ path gives a minimum-cost flow of every attained value.

Negative reverse edges prevent plain Dijkstra. Maintain vertex potentials $p[v]$ such that every residual reduced cost

$$c'(u,v) = c(u,v) + p[u] - p[v]$$

is nonnegative. Initialize potentials with Bellman–Ford distances when input costs may be negative. After Dijkstra, set $p[v] += \text{dist}[v]$ for reached vertices. The triangle inequality proves that reduced costs stay nonnegative; edges on the chosen shortest path have reduced cost zero in both residual directions after reversal.

4 Modeling quotas and convex costs

A unit-capacity worker edge enforces “at most once”; sending exactly n units enforces “exactly once.” Project lower and upper quotas are bounded edges.

If using a min-cost-flow implementation without lower bounds, lower slots may receive a reward $-BIG$. Choose BIG larger than the complete range of real objectives, then verify that all rewarded slots were filled. This is a lexicographic reduction, not a heuristic.

A separable convex cost can be expanded into increasing marginal costs. Since

$$a \cdot x^2 + b \cdot x = \sum_{k=1}^x (a \cdot (2 \cdot k - 1) + b),$$

create one unit-capacity edge for every marginal unit when capacities are small enough.

5 Worked trace: lower bounds create a routing problem

Take mandatory edges $1 \rightarrow 2$ with lower bound 3 and $2 \rightarrow 3$ with lower bound 1. After sending the mandatory parts, the balances are

```
vertex 1: -3  
vertex 2: +2  
vertex 3: +1
```

Positive balance means excess mandatory inflow; that vertex must send the excess through optional residual capacity. Negative balance needs inflow. Therefore the auxiliary network sends two units from vertex 2 and one from vertex 3 toward vertex 1. The super-source and super-sink merely expose these requirements as an ordinary flow problem.

Write one edge's balance update on paper before coding all edges. Most lower-bound bugs are one global sign reversal that still passes balanced samples.

6 Worked trace: why a residual reverse edge matters

Suppose the first unit uses an apparently cheapest route containing edge $a \rightarrow b$. After that unit, a later path may traverse the residual edge $b \rightarrow a$ with cost $-\text{cost}(a,b)$. Combining both augmentations cancels the first use of $a \rightarrow b$ and replaces it by a different assignment. Without the reverse edge, the first augmentation would be irrevocable and “cheapest path now” would be a greedy algorithm rather than min-cost flow.

Potentials do not change which path is cheapest. For any $s \rightarrow t$ path,

$$\text{sum reduced_cost} = \text{sum original_cost} + \text{potential}[s] - \text{potential}[t].$$

The final two terms are the same for every path. Reweighting exists only to make all residual edge weights nonnegative so Dijkstra is legal.

7 Network-design checklist

Statement phrase	Typical network feature
each item used at most once	unit capacity
choose exactly F objects	send exactly F units
at least / at most a quota	lower / upper bound
select one option per worker	worker node with one unit of supply
cost of the j -th use changes	parallel marginal-cost edges

After drawing a network, prove a two-way correspondence: every feasible object must map to an integral flow, and every integral flow must decode to a valid object. Checking only one direction can hide flows that exploit unintended cycles or skip a mandatory choice.

8 C++ and Python implementation notes

Store residual edges by indices into adjacency lists; adding a forward edge must set both reverse indices before either vector changes again. In C++, use `long long` for distances and total cost, and test `distance != INF` before adding potentials. Python implementations should use adjacency lists of mutable edge records; tuples make residual capacity updates awkward and object-heavy class instances can exceed memory.

If all original positive-capacity costs are nonnegative, zero initial potentials are feasible. If negative-cost edges initially have capacity, run Bellman–Ford once or otherwise establish feasible potentials. Negative reverse costs alone are safe after an augmentation because the potential update makes used shortest-path edges have zero reduced cost in both directions.

9 Proof habits

Separate three claims that are often blurred together:

1. the lower-bound transformation preserves feasibility;
2. the residual shortest-path algorithm preserves minimum cost for the amount already sent;
3. the problem-specific network is equivalent to the original choices.

A correct min-cost-flow template cannot repair a wrong modeling bijection.

10 Complexity and failure modes

Dinic gives the usual maximum-flow bounds. Successive shortest augmenting path with binary-heap Dijkstra costs $O(F * E \log V)$ when at most F unit augmentations occur.

Common errors are reversing the signs of balances, omitting the $t \rightarrow s$ return edge, applying Dijkstra before potentials are feasible, updating potentials for unreachable vertices, or choosing BIG without proving a global objective bound.

11 Exercises

A isolates bounded-circulation feasibility. B adds costs, residual repair, and potentials. C hides both quota feasibility and optimization inside assignment. D is blind and combines lower bounds with a cost objective. Derive each network on paper before reading its editorial.